

Software/Firmware Integration Readiness

Senior Design B – Integration Milestone

1. Overview of Software/Firmware Architecture

1.1 System Overview

The irr-dev firmware is implemented on ESP-IDF targeting ESP32-S3 and is organized as a modular embedded control system for sensor-driven pump operation and safety monitoring. The software integrates dual-temperature sensing with TMP117 devices over I2C, current and power monitoring with INA226 over I2C, pump control in both manual and automatic modes, and protection behavior for overtemperature, sensor fault, and overcurrent conditions. At the architectural level, the firmware is divided into coordinated functional blocks, including a main application coordinator that initializes subsystems and executes the top-level state machine, a peripheral interface layer for sensor communication and threshold detection, a control core for periodic monitoring and safety actions, a state manager for mode transitions, and startup/diagnostic logic for readiness checks.

2. Complete Code Operation Description

2.1 System Startup Sequence

Upon power-up, the firmware initializes the mode manager and system monitor, configures TMP117 sensing and INA226 monitoring interfaces, initializes the pump control interface, and registers callbacks for sensor error, overheat, and high-current events. The system then applies configured current and power thresholds, executes startup diagnostics, and transitions into the default manual operating state. If critical conditions are detected during startup or early runtime, the firmware transitions into protective modes, specifically COOLDOWN or ERROR, and enforces pump-off behavior to maintain system safety.

2.2 Operating Modes (State Diagram Description)

The firmware operates as a finite state machine with distinct operational modes. In Startup mode, initialization and diagnostics are performed before normal operation is enabled. In Manual Low, Manual Medium, and Manual High modes, pump behavior follows the selected manual level. In Automatic mode, a periodic task polls temperature, current, and power, then updates monitor logic and applies safety decisions. In Cooldown mode, the pump is disabled while thermal recovery is evaluated at defined intervals, and the system returns to manual operation when conditions become safe. In Error mode, the pump remains off while the system holds in a protected state pending reset or recovery handling. Transitions among these modes are driven by software events, including threshold crossings and callback-triggered fault conditions.

Process Diagram (Top-Level)

[Use Mermaid diagram in written submission if needed]

At a high level, execution flows from boot to subsystem initialization, into startup diagnostics, and then into the main state machine loop. From that loop, behavior branches to manual, automatic, cooldown, or error handling

as required. During automatic operation, the sensor task runs at a periodic interval of 500 milliseconds to refresh measurements and update the system monitor. Safety events originating from monitor thresholds and callback paths force transitions into cooldown or error modes when unsafe conditions are detected.

State Diagram Summary

The mode transition pattern proceeds from Startup to Manual Low after diagnostics complete. Manual modes cycle according to user selection logic, and automatic mode is entered when that behavior is enabled.

Automatic mode transitions to Cooldown on overtemperature and to Error on sensor fault or current fault.

Cooldown returns to Manual Low after thermal recovery criteria are met, while Error remains a protected hold state until reset or defined recovery handling is performed.

3. User Interface (UI) Implementation

The user interface for this milestone is implemented primarily through hardware indicators and controls, specifically LED status signaling and button-driven user input. Due to shipping delays and the absence of a fabricated production PCB during this milestone window, full physical interface validation on the final hardware stack could not be completed.

In this implementation, LEDs are designed to provide immediate state feedback during startup, diagnostics, normal operation, and fault handling, while buttons are the intended operator input path for mode-selection and reset-oriented interaction flow. UI behavior is tied directly to mode-manager transitions so that user-visible indication remains deterministic and consistent with internal control state. Validation completed in this milestone focused on software-level verification of mode transitions, callback-driven state changes, and non-blocking task behavior, with physical LED/button actuation testing deferred until the target PCB is available.

3.1 User Interaction Model

System interaction with the user is state-driven and safety-prioritized.

During startup, the user is provided initialization and diagnostic status. During manual operation, the user controls or observes pump operating level and current mode. During automatic operation, the user enables autonomous behavior while firmware continuously performs sensor-based monitoring and control. During protection events, firmware automatically enforces safe behavior by disabling the pump and presenting cooldown or error status to the user. During recovery, once safe conditions are re-established, the system transitions back to normal operation and updates the displayed state accordingly.

During startup, the user is provided initialization and diagnostic status through defined LED pattern behavior. During manual operation, the user interacts with the system through button-controlled mode selection while observing LED-confirmed operating state. During automatic operation, the user enables autonomous control and the firmware continues sensor-based monitoring and safety enforcement in the background. During protection events, the firmware automatically disables the pump and presents cooldown or error conditions through interface indicators. During recovery, once safe conditions are restored, the system returns to normal operation and updates LED-based status accordingly.

This interaction model ensures clear operator awareness while preserving automatic, deterministic protection behavior.

4. Database / Server / Remote Access (If Applicable)

Database, server, and remote-access functionality are not applicable for this irr-dev milestone. The current integration scope is focused on local embedded control, peripheral communication over I2C, and deterministic safety behavior within the firmware runtime.

4.1 Remote Sensor Access Validation

Although cloud/server access is not required in this milestone, the firmware does implement and validate software access to remote external sensors on the system bus, including an INA226 energy-monitoring device and dual TMP117 temperature devices. The software initializes I2C bus and device handles, configures sensor registers, and performs repeated runtime read transactions during automatic operation. For INA226, firmware reads bus voltage, shunt voltage, current, and power values through register-level transactions and applies threshold checks to trigger callback-driven safety actions. For TMP117, firmware accesses two independently addressed remote sensors and validates readings against configured bounds, including overtemperature and consecutive-error handling paths.

This integration demonstrates that software can successfully communicate with physically external sensing devices, retrieve operational telemetry, and use those values in real-time control logic. The automatic-mode control task polls these sensors at a fixed period, updates monitor state, and enforces protective mode transitions when unsafe conditions are detected. As a result, remote sensor access in the irr-dev codebase is not only implemented but actively integrated into the live control and safety loop.

5. Validation Procedures and Results

Validation activities to date include ESP-IDF build and compile verification against the ESP32-S3 target configuration, component integration checks across sensing, monitoring, mode control, and pump-control modules, and runtime control-path review for startup flow, mode transitions, and callback/event behavior. Subsystem readiness has been demonstrated for initialization sequencing, dual TMP117 measurement and callback paths, INA226 current and power measurement with high-current callback support, complete startup/manual/automatic/cooldown/error state-machine behavior, periodic automatic task execution, and startup diagnostics framework operation with defined placeholders for expanded hardware-depth testing.

5.1 Hardware Validation Constraint (Shipping Delay Impact)

The integration objective requires demonstration that all software/firmware is developed or procured and tested using demo/eval boards or target hardware where available. For this milestone, the software/firmware implementation objective has been met; however, full physical test execution was constrained by hardware availability. Specifically, shipping delays and the lack of an assembled production PCB prevented physical end-to-end validation on the final interface hardware.

As a result, testing completed in this phase is software-centric and includes successful build verification, module-level integration checks, sensor communication-path validation in code, and control/state-machine verification. Physical validation items pending hardware arrival include LED indication behavior on final board wiring, button input debounce and transition behavior on real GPIO paths, and full pump-control interaction tests on the integrated PCB assembly. These activities are planned as the next execution step immediately after board receipt and bring-up.

5.2 Software-Only System Test Execution and Logged Outputs

To demonstrate subsystem integration in the absence of physical hardware, a software-only test harness was executed against the irr-dev codebase. This validation used static code-path checks and integration assertions to confirm that major firmware subsystems are connected and operable from a software architecture perspective. Test execution and outputs were logged in `software_only_system_tests.log`, generated by `software_only_system_tests.py`.

Five system-level software tests were executed. Test T1 (Bootstrap Integration) passed by confirming that all core modules are initialized through the startup chain, including mode manager, system monitor, TMP117 interface, INA226 interface, and pump interface. Test T2 (Safety Callback Wiring) passed by confirming callback registration and fault-to-mode transitions for sensor error, overtemperature, and high-current events. Test T3 (State-Machine Coverage) passed by confirming that all required runtime modes are implemented in the primary control switch path. Test T4 (Automatic Control Loop) passed by confirming that the automatic task reads remote sensors, updates system monitor logic, and runs at the defined periodic interval. Test T5 (Remote Sensor Access Path) passed by confirming register-transaction-based access paths for INA226 and dual-address TMP117 devices.

All five software-only system tests passed (5/5), providing objective evidence that subsystem structure, control flow, and safety-event wiring are implemented and internally consistent even though physical board-level verification remains pending.

Summary

The software/firmware architecture is integration-ready at the control-flow and safety-event level, and core functionality has been implemented and verified in software. The primary open risk is schedule-driven hardware validation delay caused by unavailable physical PCB hardware. Remaining work is focused on executing deferred physical interface tests (LEDs, buttons, and integrated I/O behavior) as soon as hardware is available, followed by full hardware validation demonstration.